Standard Tipografici ed Architettura Operativa per la Visualizzazione Dati di Laboratorio

Manuale Tecnico e Guida alle API dei Moduli `plot_typo` e `chi_squared_2d_map`

Laboratorio di Fisica Sperimentale e Computazionale

22 ottobre 2026

Sommario

Questo manuale definisce le specifiche implementative, le convenzioni tipografiche e i protocolli d'uso del modulo grafico `plot_typo.py` e la sua interazione con i grafici di laboratorio. L'infrastruttura standardizza la produzione di figure scientifiche pronte per la pubblicazione su riviste internazionali (es. *Physical Review*, *European Physical Journal*). Vengono documentate in dettaglio: l'integrazione di LaTeX reale con font Palatino (`mathpazo`), la propagazione globale dei parametri scalari tramite il dizionario `TYPO_CONFIG`, la gestione scalare delle unità con `format_sci_axis`, il controllo indipendente delle scale lineari e logaritmiche con tacche bidirezionali `inout` e soppressione flessibile delle tacche minori in `apply_academic_ticks_and_grid`.

Indice

1 Introduzione e Specifiche Tipografiche **2**
1.1 Standard Tipografico Palatino LaTeX 2

2 Inizializzazione Globale e Dizionario `TYPO_CONFIG` **2**

3 Normalizzazione Scientifica degli Assi: `format_sci_axis` **4**
3.1 Regola di Chiamata Critica 4

4 Geometria degli Assi, Tacche e Griglie **4**
4.1 Configurazione con `apply_academic_ticks_and_grid` 4
4.2 Routine Rapida `disable_minor_ticks` 5

5 Topologia dei Layout di Pubblicazione **6**
5.1 Grafico a Due Pannelli Accostati (Fit + Residui) 6
5.2 Standard per Barre d'Errore e Legende 6

1 Introduzione e Specifiche Tipografiche

La presentazione visiva di misure fisiche richiede lo stesso rigore matematico applicato alla modellizzazione teorica e all'analisi statistica. Nelle librerie grafiche ad alto livello si riscontrano spesso difetti tipografici:

1. **Disomogeneità dei font**: il testo degli assi usa font sans-serif di sistema, mentre le espressioni matematiche usano font TeX standard (Computer Modern), generando contrasti sgradevoli.
2. **Cifre ridondanti e troncamenti arbitrari**: valori grandi o piccoli vengono stampati con molti zeri continui (es. $90000, 105000$) oppure le etichette alternano cifre intere e decimali misti ($1, 1.05, 1.1$), violando la coerenza di lettura.
3. **Tacche orientate verso l'interno o asimmetriche**: le tacche `inside` entrano nel canvas oscurando i dati periferici, mentre le tacche `outside` collidono con le etichette numeriche.
4. **Pannelli residui disgiunti**: l'inserimento di fessure vuote tra il pannello del fit e quello dei residui disperde l'allineamento orizzontale punto-per-punto.

1.1 Standard Tipografico Palatino LaTeX

La libreria adotta come riferimento il carattere **Palatino**:

- **Testo descrittivo, legende e titoli**: Palatino tondo e corsivo;
- **Formule, grandezze fisiche e simboli greci**: font matematico `mathpazo`;
- **Unità di misura**: racchiuse in carattere tondo (`\mathrm{...}`), posizionate in ambiente matematico delimitato da parentesi quadre.

2 Inizializzazione Globale e Dizionario `TYPO_CONFIG`

Il modulo definisce a livello globale il dizionario `TYPO_CONFIG`, che memorizza le grandezze tipografiche condivise istantaneamente tra tutti i moduli (incluso `chi_squared_2d_map`):

```
TYPO_CONFIG = {
    "textsize": 11.0,
    "labelsize": 11.5,
    "ticksize": 10.0,
    "legendsize": 9.5,
    "titlesize": 11.5,
    "major_tick_len": 6.0,
    "major_tick_width": 0.8,
    "minor_tick_len": 3.5,
    "minor_tick_width": 0.5,
}
```

Listing 1: Configurazione standard globale in `plot_typo.py`.

La funzione `setup_academic_style()` interviene sia su `matplotlib.rcParams` forzando la compilazione TeX sia aggiornando sul posto (*in-place*) il dizionario `TYPO_CONFIG`:

```
def setup_academic_style(
    textsize: float = 11.0,
    labelsize: float = 11.5,
    ticksize: float = 10.0,
    legendsize: float = 9.5,
    titlesize: float = 11.5,
    major_tick_len: float = 6.0,
    major_tick_width: float = 0.8,
```

```
    minor_tick_len: float = 3.5,
    minor_tick_width: float = 0.5,
) -> None:
    plt.rcParams.update({
        "text.usetex": True,
        "font.family": "serif",
        "font.serif": ["Palatino", "URW Palladio L", "TeX Gyre Pagella"],
        "text.latex.preamble": r"""
            \usepackage{amsmath}
            \usepackage{amssymb}
            \usepackage{mathpazo}
        """,
        "font.size": textsize,
        "axes.labelsize": labelsize,
        "xtick.labelsize": ticksize,
        "ytick.labelsize": ticksize,
        "legend.fontsize": legendsize,
        "figure.titlesize": titlesize,
    })

    TYPO_CONFIG.update({
        "textsize": textsize,
        "labelsize": labelsize,
        "ticksize": ticksize,
        "legendsize": legendsize,
        "titlesize": titlesize,
        "major_tick_len": major_tick_len,
        "major_tick_width": major_tick_width,
        "minor_tick_len": minor_tick_len,
        "minor_tick_width": minor_tick_width,
    })
```

Listing 2: Inizializzazione dello stile Palatino LaTeX e sincronizzazione dinamica.

3 Normalizzazione Scientifica degli Assi: `format_sci_axis`

La funzione `format_sci_axis` risolve l'estrazione scalare del fattore $\times 10^n$, assicurando che le tacche numeriche rimangano pulite e omogenee:

```python
def format_sci_axis(
    ax: plt.Axes,
    vals: Sequence[float] | np.ndarray,
    base_label: str,
    axis: str = "y",
    unit: str = "",
    min_exp: int = 2,
    dec: int = 2,
) -> str:
    arr = np.asarray(vals, dtype=float)
    non_zero = arr[arr != 0]
    amax = np.max(np.abs(non_zero)) if len(non_zero) > 0 else 0.0
    esp = int(math.floor(math.log10(amax))) if amax > 0 else 0

    u_clean = unit.strip().replace("$", "")
    if u_clean.startswith(r"\mathrm{") and u_clean.endswith("}"):
        u_clean = u_clean[len(r"\mathrm{"):-1]

    unit_math = rf"\mathrm{{{u_clean}}}" if u_clean else ""

    if abs(esp) >= min_exp:
        scale = 10.0**esp
        fmt = ticker.FuncFormatter(lambda v, _: f"{v / scale:.{dec}f}")
        getattr(ax, f"{axis}axis").set_major_formatter(fmt)

        if unit_math:
            suffix = rf" [$\times 10^{{{esp}}}\ {unit_math}$]"
        else:
            suffix = rf" [$\times 10^{{{esp}}}$]"
        return f"{base_label}{suffix}"

    suffix = rf" [${unit_math}$]" if unit_math else ""
    return f"{base_label}{suffix}"
```

Listing 3: Implementazione di `format_sci_axis`.

3.1 Regola di Chiamata Critica

Regola Operativa: Poiché i localizzatori di tacche come `AutoMinorLocator` o `MaxNLocator` reimpostano i formattatori numerici, la chiamata a `format_sci_axis()` deve avvenire sempre **dopo** l'invocazione di `apply_academic_ticks_and_grid()`.

4 Geometria degli Assi, Tacche e Griglie

4.1 Configurazione con `apply_academic_ticks_and_grid`

La routine `apply_academic_ticks_and_grid` standardizza il telaio visivo, introducendo il supporto al controllo selettivo delle suddivisioni secondarie:

```python
def apply_academic_ticks_and_grid(
    ax: plt.Axes,
    scale_x: str = "linear",  # "linear" oppure "log"
    scale_y: str = "linear",  # "linear" oppure "log"
    show_minor: bool = True,  # Abilita o sopprime le tacche minori
    major_len: float = 8.0,
    minor_len: float = 4.0,
```

```
    major_width: float = 0.8,
    minor_width: float = 0.5,
) -> None:
```

Listing 4: Firma e logica di `apply_academic_ticks_and_grid`.

- **Tacche bidirezionali** `inout`: le tacche attraversano il bordo dell'asse centrandosi esattamente sul perimetro esterno ed interno, evitando interferenze con testi e dati.
- **Parametro** `show_minor`: se `True`, imposta `AutoMinorLocator` (o `LogLocator` con sottomultipli) e attiva la griglia secondaria (`ls=":"`, color `#e5e5e5`). Se `False`, assegna `NullLocator()` azzerando lunghezza, spessore e griglia minore.
- **Disaccoppiamento Lineare/Logaritmico**: assi indipendenti con formattatore logaritmico automatico `LogFormatterMathtext()`.

4.2 Routine Rapida `disable_minor_ticks`

Per rimuovere istantaneamente tacche minori e griglia secondaria su grafici ad alta densità di punti o pannelli ridotti, è disponibile la funzione:

```
def disable_minor_ticks(ax: plt.Axes) -> None:
    ax.xaxis.set_minor_locator(ticker.NullLocator())
    ax.yaxis.set_minor_locator(ticker.NullLocator())
    ax.tick_params(axis="both", which="minor", length=0, width=0,
                   bottom=False, top=False, left=False, right=False)
    ax.grid(False, which="minor")
```

5 Topologia dei Layout di Pubblicazione

5.1 Grafico a Due Pannelli Accostati (Fit + Residui)

La visualizzazione standard di laboratorio accosta il pannello della regressione a quello inferiore dei residui azzerando lo spazio verticale (`hspace=0.0`):

```
fig = plt.figure(figsize=(6.0, 5.2))
gs = gridspec.GridSpec(2, 1, height_ratios=[3.2, 1.0])
gs.update(hspace=0.0)

ax_main = fig.add_subplot(gs[0, 0])
ax_res  = fig.add_subplot(gs[1, 0], sharex=ax_main)

# Soppressione etichette asse X sul pannello superiore
ax_main.tick_params(axis="x", labelbottom=False)

# Rimuove il tick inferiore su ax_main per prevenire collisioni col bordo di giunzione
ax_main.yaxis.set_major_locator(ticker.MaxNLocator(nbins=5, prune="lower"))

# Configurazione tacche e formattazione scientifica
apply_academic_ticks_and_grid(ax_main, scale_x="linear", scale_y="linear")
apply_academic_ticks_and_grid(ax_res, scale_x="linear", scale_y="linear")

ax_main.set_ylabel(format_sci_axis(ax_main, y_curve, r"Intensit\'a", axis="y", unit="u.a."))
ax_res.set_ylabel(format_sci_axis(ax_res, residui, r"Residui", axis="y", unit="u.a.", dec=1))
ax_res.set_xlabel(format_sci_axis(ax_res, x_data, r"Frequenza di scansione", axis="x", unit="Hz"))
```

Listing 5: Costruzione del layout accostato con residui.

5.2 Standard per Barre d'Errore e Legende

- **Barre d'errore pulite**: utilizzare `capsize=0` (o `capsize=1.6`) associato a `capthick=0.8` ed `elinewidth=0.8`.
- **Legenda a cornice opaca**:

```
ax.legend(
    loc="upper right", frameon=True, fancybox=False,
    facecolor="white", edgecolor="black", framealpha=1.0,
    borderpad=0.4, handlelength=1.4
)

```